



Nexus Code Starter Kit

Uma equipe inteira de especialistas de software, pronta em segundos dentro do seu projeto.

22 Agentes

15 Comandos

43 Skills

Spec-driven validável



Gate de qualidade

```
npx nexus-code-starter-kit
```

Apresentação Completa · Versão 1.11.0 · Licença MIT · por gabrielbicca

Uma “caixa de ferramentas” pronta para o Claude Code. Você instala em segundos e o seu assistente de programação ganha uma **equipe inteira de especialistas, atalhos de comando e manuais de boas práticas** — tudo organizado dentro do seu projeto.

Versão: 1.11.0 · Licença: MIT · Autor: gabrielbicca

Índice

1. O que é, em uma frase
2. Explicando como se você nunca tivesse programado
3. Que problema isso resolve?
4. Como instalar (3 passos)
5. Os 4 pilares do kit
6. Os 22 Agentes — sua equipe de especialistas
7. Os 15 Comandos — atalhos do dia a dia
8. As 43 Skills — a biblioteca de conhecimento
9. A base de conhecimento (docs/) — o “cérebro” do projeto
10. Contexto e scripts de verificação
11. Um dia de trabalho usando o kit
12. Atualizando o kit
13. Glossário para leigos
14. Perguntas frequentes (FAQ)

1. O que é, em uma frase

O **Nexus Code Starter Kit** é um pacote que turbinha o **Claude Code** (o programador de inteligência artificial da Anthropic) com **22 especialistas virtuais, 15 comandos rápidos, 43 manuais de boas práticas**, uma **estrutura de documentação organizada com validação automática** e um **gate de qualidade obrigatório** (testes + segurança + verificação com evidência) — instalado dentro de qualquer projeto com **um único comando**.

2. Explicando como se você nunca tivesse programado

Imagine que você contratou **um assistente de IA muito inteligente** para construir seu site, aplicativo ou automação. Ele é ótimo — mas, sozinho, é como um funcionário novo que **não conhece sua empresa e faz tudo do seu jeito improvisado**.

O Nexus Code Starter Kit é como dar a esse assistente:

-  **Uma equipe de especialistas** — em vez de uma pessoa fazendo tudo, ele pode “chamar” o especialista certo para cada tarefa (o de banco de dados, o de design, o de segurança...).
-  **Botões de atalho** — em vez de explicar tarefas longas toda vez, você aperta um “botão” (ex.: “faça o deploy”, “crie testes”) e ele já sabe o passo a passo.
-  **Manuais de boas práticas** — para ele não inventar, e sim seguir o que o mercado considera correto.
-  **Um arquivo organizado** — onde tudo que foi decidido fica registrado, para nada se perder.

Resultado: o assistente trabalha **mais rápido, mais organizado e com muito mais qualidade** — como uma empresa de software de verdade, em vez de um freelancer improvisando.

3. Que problema isso resolve?

Sem o kit	Com o kit
A IA faz tudo “genérico”, sem padrão.	Cada tarefa vai para um especialista .
Você repete as mesmas instruções longas.	Você usa comandos curtos prontos.
Decisões importantes se perdem no chat.	Tudo fica documentado em <code>docs/</code> .
Fácil cometer erros de segurança/qualidade.	Há audidores e manuais integrados.
Cada projeto começa do zero.	Você tem uma fundação pronta em segundos.

 **A grande ideia: “spec-driven”.** O kit segue a filosofia de que **a documentação vem antes do código**. Primeiro se descreve *o que* será feito (a “spec”), depois se constrói. Isso evita retrabalho e mantém todo mundo (humanos e IA) na mesma página.

4. Como instalar (3 passos)

Pré-requisitos: ter o [Node.js 18+](#) e o [Claude Code](#) instalados.

Passo 1 — Abra a pasta do seu projeto no terminal

Passo 2 — Rode um único comando

```
npx nexus-code-starter-kit
```

Ou, no Windows (PowerShell):

```
irm https://raw.githubusercontent.com/gabrielbicca/nexus-code-starter-kit/main/install.ps1 | iex
```

Passo 3 — Pronto!

O kit cria a pasta `.claude/` (com agentes, comandos e skills) e a pasta `docs/` (a base de conhecimento). É **seguro**: se já existir algo, ele **preserva** seus arquivos e só adiciona o que falta.

```
.claude/
├─ agents/      → 22 especialistas
├─ commands/    → 15 atalhos de comando
├─ skills/       → 43 manuais de conhecimento
├─ scripts/     → verificadores automáticos
└─ context/     → blocos de contexto reutilizáveis

docs/           → documentação do projeto (specs, decisões, diários)
CLAUDE.md       → o “manual do projeto” para a IA (opcional)
```

5. Os 4 pilares do kit

NEXUS CODE STARTER KIT			
 AGENTES	 COMANDOS	 SKILLS	 DOCS
22 espe- cialistas virtuais	15 atalhos rápidos	43 manuais de boas práticas	base de conhecimento spec-driven

- 1. **Agentes** — *quem* faz o trabalho (especialistas).
- 2. **Comandos** — *como* você pede o trabalho (atalhos).
- 3. **Skills** — *o conhecimento* que garante qualidade.
- 4. **Docs** — *a memória* de tudo que foi decidido — **com validação automática** que confere se código e documentação continuam coerentes.

6. Os 22 Agentes — sua equipe de especialistas

Pense em cada agente como **um funcionário sênior** com uma especialidade. Você os chama escrevendo `@nome-do-agente`. Eles estão agrupados abaixo por área.

Coordenação e planejamento

Agente	O que faz (em linguagem simples)
@orchestrator	O maestro . Quando a tarefa é grande e envolve várias áreas, ele divide o trabalho e aciona os outros especialistas na ordem certa.
@project-planner	O planejador . Pega uma ideia vaga (“quero um app de pedidos”) e transforma num plano claro, com etapas e dependências.

Produto e requisitos

Agente	O que faz (em linguagem simples)
@product-manager	Traduz o que os usuários precisam em funcionalidades bem descritas (histórias, critérios, prioridades).
@product-owner	Define o mínimo necessário para lançar (o MVP) e faz as escolhas difíceis entre escopo, prazo e qualidade.

Construção (desenvolvimento)

Agente	O que faz (em linguagem simples)
@backend-specialist	Constrói a parte invisível do sistema: regras de negócio, APIs, integrações, login.
@frontend-specialist	Constrói a parte visível : telas, botões, formulários (React, Next.js, Tailwind).
@database-architect	Cuida do banco de dados : como os dados são guardados, organizados e protegidos.
@mobile-developer	Faz aplicativos de celular (iOS e Android, React Native, Flutter).
@game-developer	Faz jogos (Unity, Godot, Phaser, Three.js): física, fases, pontuação.
@n8n-specialist	Cria automações que conectam apps (n8n): webhooks, chatbots, robôs de tarefas.

Qualidade e investigação

Agente	O que faz (em linguagem simples)
@debugger	O detetive de bugs . Descobre a causa raiz quando algo quebra ou se comporta de forma estranha.
@test-engineer	Escreve testes que verificam se cada peça do código funciona como deveria.
@qa-automation-engineer	Cria testes automáticos de ponta a ponta que simulam um usuário usando o sistema.
@performance-optimizer	O especialista em velocidade . Deixa o sistema mais rápido e leve.
@explorer-agent	O mapeador . Explora um projeto desconhecido e responde “onde está o quê?”.
@code-archaeologist	O arqueólogo . Entende código antigo e bagunçado e planeja como melhorá-lo com segurança.
@clean-code-auditor	O faxineiro técnico . De tempos em tempos, varre o projeto inteiro atrás de código morto e débito técnico (funções nunca chamadas, imports órfãos, código comentado sem contexto) e entrega um plano de refatoração priorizado.

Segurança

Agente	O que faz (em linguagem simples)
@security-auditor	O auditor . Revisa o código procurando falhas de segurança (defensivo).
@penetration-tester	O hacker do bem . Tenta invadir o sistema (em ambiente autorizado) para achar brechas antes dos criminosos.

Operações e crescimento

Agente	O que faz (em linguagem simples)
@devops-engineer	Cuida de colocar o sistema no ar (deploy, Docker, CI/CD). ⚠ Mexe em produção.
@seo-specialist	Faz seu site aparecer no Google e ser citado por IAs (SEO e GEO).
@documentation-writer	Escreve documentação clara: READMEs, manuais, tutoriais, guias.

7. Os 15 Comandos — atalhos do dia a dia

Comandos começam com `/`. São **botões prontos** para tarefas comuns — você não precisa explicar o passo a passo.

Comando	O que faz (em linguagem simples)
<code>/spec</code>	Cria uma especificação numerada (SPEC-001, SPEC-002...) de uma nova funcionalidade. <i>Documente antes de codar.</i>
<code>/adr</code>	Registra uma decisão de arquitetura importante (ADR) e o porquê dela.
<code>/plan</code>	Gera um plano de projeto detalhado (só o plano, sem escrever código ainda).
<code>/brainstorm</code>	Faz uma chuva de ideias estruturada, explorando opções antes de decidir.
<code>/create</code>	Cria um aplicativo novo do zero, conversando com você sobre o que precisa.
<code>/enhance</code>	Adiciona ou melhora funcionalidades em um app que já existe.
<code>/orchestrate</code>	Aciona o maestro para coordenar vários especialistas numa tarefa complexa.
<code>/test</code>	Cria e roda testes do seu código.
<code>/verify</code>	Verifica o projeto : qualidade + coerência da documentação (spec-driven). Modo rápido (no dia a dia) ou completo (antes de publicar).
<code>/debug</code>	Liga o modo investigação para resolver um problema de forma sistemática.
<code>/deploy</code>	Publica o sistema com verificações de segurança antes.
<code>/preview</code>	Liga/desliga o servidor local para você ver o projeto rodando.
<code>/status</code>	Mostra o painel de status do projeto e dos agentes.
<code>/ui-ux-pro-max</code>	Inteligência de design : 50+ estilos visuais, 95+ paletas de cores e geração de design system.
<code>/dotnet-new</code>	Gera um projeto backend .NET do zero (Clean Architecture + DDD ou N-Tier, sempre versões LTS).

8. As 43 Skills — a biblioteca de conhecimento

As **skills** são **manuals de boas práticas** que os agentes consultam automaticamente. Você raramente precisa chamá-las — elas garantem que a IA **siga o padrão certo** em vez de improvisar. Estão agrupadas por tema abaixo.

Planejamento, arquitetura e coordenação

- **architecture** — como tomar decisões de arquitetura e avaliar trade-offs.
- **plan-writing** — como escrever planos claros com etapas e dependências.
- **brainstorming** — protocolo de perguntas para clarear requisitos confusos.
- **behavioral-modes** — modos de trabalho da IA (planejar, implementar, revisar...).
- **parallel-agents** — como rodar vários agentes em paralelo.
- **intelligent-routing** — escolher automaticamente o especialista certo.
- **mcp-builder** — como construir servidores MCP (extensões de ferramentas para IA).

Construção de aplicações e back-end

- **app-builder** — o orquestrador que monta apps completos do zero.
- **api-patterns** — como desenhar APIs (REST, GraphQL, tRPC), versões, paginação.
- **database-design** — desenho de banco de dados, índices, escolha de ORM.
- **nodejs-best-practices** — boas práticas de Node.js.
- **python-patterns** — boas práticas de Python.
- **rust-pro** — Rust moderno, async e sistemas de alto desempenho.
- **dotnet-backend-standards** — padrão de backend .NET do kit (Clean Architecture + DDD, segurança, pt-BR).
- **dotnet-orm-efcore** — convenções de Entity Framework Core (escritas, migrations).
- **dotnet-orm-dapper** — convenções de Dapper (leituras complexas, relatórios, SQL de alta performance).
- **dotnet-project-scaffold** — estrutura de projeto .NET novo (Clean Architecture ou N-Tier, versões LTS).

Front-end e design

- **frontend-design** — pensamento de design para telas web.
- **tailwind-patterns** — padrões do Tailwind CSS v4.
- **nextjs-react-expert** — otimização de performance em React/Next.js (da engenharia da Vercel).
- **mobile-design** — design mobile-first para iOS e Android.
- **ui-ux-pro-max** — inteligência de design com estilos, cores e tipografia.
- **web-design-guidelines** — revisão de UI contra diretrizes de acessibilidade e UX.
- **i18n-localization** — internacionalização (vários idiomas, RTL).

Qualidade, testes e depuração

- **tdd-workflow** — desenvolvimento guiado por testes (ciclo vermelho-verde-refatora).
- **testing-patterns** — padrões de testes (unitários, integração, mocks).
- **webapp-testing** — testes de aplicação web (E2E, Playwright).
- **clean-code** — código limpo, direto e sem complexidade desnecessária.
- **dead-code-sweep** — varredura periódica de código morto e débito técnico, com relatório e backlog de refatoração priorizado.
- **code-review-checklist** — checklist de revisão de código.
- **lint-and-validate** — checagem automática de qualidade e sintaxe.
- **systematic-debugging** — método de depuração em 4 fases.
- **performance-profiling** — medir e otimizar desempenho.

Segurança

- **red-team-tactics** — táticas de ataque ético (baseadas em MITRE ATT&CK).
- **vulnerability-scanner** — análise de vulnerabilidades (OWASP 2025, supply chain).

Operações e infraestrutura

- **deployment-procedures** — como publicar com segurança e fazer rollback.
- **server-management** — gestão de servidores, monitoramento, escala.
- **documentation-templates** — modelos de documentação (README, API docs).

Especialidades

- **game-development** — orquestrador de desenvolvimento de jogos.
- **seo-fundamentals** — fundamentos de SEO, E-E-A-T, Core Web Vitals.
- **geo-fundamentals** — otimização para buscadores de IA (ChatGPT, Claude, Perplexity).

Terminal e ambiente

- **bash-linux** — padrões de terminal no Linux/macOS.
- **powershell-windows** — padrões de PowerShell no Windows.

9. A base de conhecimento (docs/) — o “cérebro” do projeto

Este é o coração da filosofia **spec-driven: a documentação é o cérebro do desenvolvimento**. Tudo que é decidido fica guardado aqui, versionado junto com o código.

```
docs/
├─ README.md           → Índice (o mapa da documentação)
├─ 00_Meta/            → Modelos prontos (Feature-Spec, ADR, Migration)
├─ 01_Architecture/    → Decisões de arquitetura (ADRs) e diagramas
├─ 02_Specs/           → Especificações de funcionalidades
│   └─ Migrations/     → Documentação das mudanças de banco
├─ 03_Sprint_Logs/     → Diários de trabalho (o que foi feito em cada ciclo)
└─ 04_Assets/          → Imagens e diagramas
```

Por que isso importa para um leigo? Porque é o que impede que o conhecimento do projeto **se perca**. Se daqui a 6 meses você (ou outra pessoa) perguntar “*por que decidimos fazer assim?*”, a resposta está documentada — não esquecida em um chat antigo.

A criação é **idempotente**: se a pasta já existe, nada é apagado; só o que falta é adicionado.

10. Contexto e scripts de verificação

Contexto importável (.claude/context/)

São **blocos de contexto reutilizáveis** que o arquivo `CLAUDE.md` (o manual do projeto para a IA) pode importar **sob demanda**, mantendo o manual principal leve:

- `migrations` · `specs-adrs-pages` · `business-rules` · `github-project` · `ui-patterns` · `agents-skills`

Scripts de verificação (.claude/scripts/)

Pequenos programas que **checam a saúde do projeto** automaticamente:

- `checklist.py` — roda uma bateria de verificações de qualidade (rápido, no dia a dia).

- `spec_drift.py` — confere se documentação, banco e código estão **coerentes entre si**. Vai além de “links quebrados”: verifica se toda mudança de banco aponta para uma spec, se uma spec marcada como *concluída* tem **todos** os critérios cumpridos, se a spec aponta para o código que a implementa **e se o Gate de qualidade do kit foi cumprido** (testes implementados + review de segurança + verificação com evidência — ver abaixo).
- `verify_all.py` — verificação geral antes de publicar (a checagem spec-driven roda primeiro).
- `auto_preview.py` — liga o preview automático.
- `session_manager.py` — gerencia a sessão de trabalho.

🔵 Validação automática (opcional)

Na instalação, o kit oferece ligar um **“guarda automático”**: antes de cada salvamento no histórico (commit) e a cada envio no GitHub, ele roda a checagem spec-driven sozinho. Assim a regra *“documentação antes do código”* deixa de depender de alguém lembrar — vira parte da esteira. O comando `/verify` é o atalho para rodar isso quando quiser.

🔴 Gate de qualidade — regra obrigatória do kit

Todo desenvolvimento novo só é considerado concluído quando cumprir três exigências, sem exceção:

1. **Testes implementados na camada de testes** — *toda funcionalidade da feature precisa estar mapeada em pelo menos um teste* (o `@test-engineer` implementa; E2E com o `@qa-automation-engineer`). Nenhuma funcionalidade fica sem teste.
2. **Review de segurança executado** — o `@security-auditor` revisa a implementação de toda feature nova (não só quando mexe em login), e os apontamentos são tratados.
3. **Verificação executada com evidência** — o `/verify` (ou a suíte de testes) roda **depois** da implementação e a **saída real** (testes passando, verificação limpa) fica registrada. Concluído se declara com evidência — *“deve funcionar” não é evidência*.

Isso não é recomendação, é **regra verificável**: a SPEC tem uma seção “Gate de qualidade” com esses três itens, e o `spec_drift.py` **acusa erro** se uma spec for marcada como concluída sem eles — inclusive no commit e no CI, se o guarda automático estiver ligado.

11. Um dia de trabalho usando o kit

Veja como tudo se conecta na prática. Suponha que você queira **adicionar um sistema de login** ao seu app:

1. Você: `/spec` → cria a SPEC-001 “Sistema de Login”
2. Você: `@orchestrator` implemente a SPEC-001
3. Maestro: → `@database-architect` cria a tabela de usuários
→ `@backend-specialist` cria as rotas de login e cadastro
→ `@frontend-specialist` cria as telas de login
→ `@security-auditor` revisa tudo procurando falhas
→ `@test-engineer` escreve os testes
4. Você: `/test` → roda os testes
5. Você: `/verify` → confere qualidade + coerência spec-driven
6. Você: `/deploy` → publica com verificações

Cada especialista consulta as **skills** relevantes automaticamente, e tudo que foi decidido fica registrado em `docs/` . Você acompanha pelo `/status` .

● Repare nos passos do `@security-auditor` e do `@test-engineer` : eles **não são opcionais**. Pelo Gate de qualidade do kit, toda feature nova termina com os testes implementados (cada funcionalidade mapeada em teste), com o review de segurança executado e com a **evidência da verificação registrada** (a saída real do `/verify`) — o próprio `/verify` cobra isso.

📋 **Acompanhe cada etapa — não espere o final.** A cada passo do fluxo, confira se ele está sendo seguido de forma correta: use o `/status` para ver onde o trabalho está, rode o `/verify` ao fim de cada etapa (não só antes do deploy) e confirme na SPEC que os checkboxes correspondentes foram marcados. Se uma etapa foi pulada (ex.: implementou sem SPEC, concluiu sem testes), pare e corrija **antes** de seguir — é muito mais barato ajustar no meio do caminho do que descobrir no final.

🔧 **Plugins externos (ex.: Superpowers) complementam, não substituem.** O fluxo do projeto é sempre o do Nexus (`/spec` → `/plan` → `@orchestrator` → `/verify`). Plugins entram só como apoio pontual — TDD, code review, refatoração, debugging — sem criar specs ou planos paralelos. Em conflito de instruções, as regras do kit prevalecem.

🧹 **De tempos em tempos, faça a faxina.** Ao fim de um sprint ou antes de um release, rode o `@clean-code-auditor` : ele varre o projeto inteiro atrás de código morto e débito técnico e devolve um plano de refatoração priorizado. O `/status` lembra quando a varredura está atrasada.

O mais importante: você **não precisa saber programar** para reger essa orquestra. Você descreve o que quer, e a equipe de IA executa seguindo padrões profissionais.

12. Atualizando o kit

Para receber melhorias e correções, basta rodar o instalador de novo na mesma pasta:

```
npx nexus-code-starter-kit
```

O instalador detecta a instalação existente e oferece **3 opções seguras**:

- **Merge** — só adiciona arquivos novos (preserva os seus). *Modo padrão e seguro.*
- **Sobrescrever** — substitui todos os agentes/skills/comandos pela versão nova (para puxar correções).
- **Cancelar** — sai sem mexer em nada.

A versão instalada fica gravada em `.claude/.kit-version` , e o instalador avisa quando há uma versão mais nova disponível.

13. Glossário para leigos

Termo	O que significa, sem jargão
Claude Code	O programador de inteligência artificial da Anthropic que executa as tarefas.
Agente	Um “especialista virtual” com uma área de atuação.
Skill	Um manual de boas práticas que a IA consulta.
Comando (/)	Um atalho que dispara uma tarefa pronta.
Spec	A descrição do <i>que</i> será construído, feita antes de codar.
ADR	O registro de uma decisão de arquitetura importante e o motivo.
Deploy	Colocar o sistema no ar, disponível para os usuários.
Bug	Um erro ou defeito no software.
Back-end / Front-end	A parte “de trás” (lógica) e a parte “da frente” (telas) de um sistema.
Banco de dados	Onde as informações do sistema ficam guardadas de forma organizada.
MVP	Versão mínima funcional, suficiente para lançar e validar.
CI/CD	Esteira automática que testa e publica o código.
SEO / GEO	Aparecer bem no Google (SEO) e ser citado por IAs (GEO).
Spec-driven	Filosofia em que a documentação vem antes do código.
Idempotente	Pode rodar várias vezes sem estragar nada que já existe.

14. Perguntas frequentes (FAQ)

Isso é seguro? Vai apagar meus arquivos? Não. A instalação preserva o que já existe (modo Merge). Só a opção “Sobrescrever”, escolhida por você, troca os arquivos do próprio kit.

Funciona em qualquer projeto? Sim — projetos novos ou já existentes, em diversas linguagens e frameworks. O kit se adapta.

Funciona no Windows, Mac e Linux? Sim. Há instalador via `npx` (todos) e via PowerShell (Windows).

Quanto custa? O kit é **gratuito e open source** (licença MIT). Você só precisa do Claude Code.

Para que servem tantos agentes se a IA já é inteligente? Especializar melhora a qualidade. Um especialista focado em segurança encontra coisas que um “generalista” deixaria passar — exatamente como em uma empresa de verdade.

Sua equipe de software, pronta em segundos.

```
npx nexus-code-starter-kit
```

github.com/gabrielbicca/nexus-code-starter-kit